

```
1: from __future__ import annotations
2:
3: import threading
4: import time
5: from typing import Any
6:
7: import cv2
8: import numpy as np
9: import requests
10:
11: from .config import AppConfig
12:
13:
14: class Esp32KameraClient:
15:     def __init__(self, config: AppConfig) -> None:
16:         self.config = config
17:
18:     def frame_aufnehmen(self) -> np.ndarray:
19:         antwort = requests.get(self.config.capture_url, timeout=10)
20:         antwort.raise_for_status()
21:
22:         frame = cv2.imdecode(
23:             np.frombuffer(antwort.content, dtype=np.uint8), cv2.IMREAD_COLOR
24:         )
25:         if frame is None:
26:             raise RuntimeError(
27:                 f"JPEG von {self.config.capture_url} konnte nicht dekodiert werden."
28:             )
29:         return frame
30:
31:     def stream_oeffnen(self) -> Any:
32:         backends = []
33:         ffmpeg_backend = getattr(cv2, "CAP_FFMPEG", None)
34:         if ffmpeg_backend is not None:
35:             backends.append(ffmpeg_backend)
36:         backends.append(None)
37:
38:         for backend in backends:
39:             if backend is None:
40:                 video_capture = cv2.VideoCapture(self.config.stream_url)
41:             else:
42:                 video_capture = cv2.VideoCapture(self.config.stream_url, backend)
43:
44:             video_capture.set(cv2.CAP_PROP_BUFFERSIZE, 1)
```

esp32_1.0.py | Seite 2

```
45:         if video_capture.isOpened():
46:             return video_capture
47:
48:         video_capture.release()
49:
50:     return cv2.VideoCapture(self.config.stream_url)
51:
52:     def buzzer_impuls(self, dauer_ms: int) -> None:
53:         parameter = {"state": "on"}
54:         if dauer_ms > 0:
55:             parameter["duration_ms"] = int(dauer_ms)
56:
57:         antwort = requests.get(self.config.buzzer_url, params=parameter, timeout=2)
58:         antwort.raise_for_status()
59:
60:     def buzzer_stoppen(self) -> None:
61:         antwort = requests.get(
62:             self.config.buzzer_url, params={"state": "off"}, timeout=2
63:         )
64:         antwort.raise_for_status()
65:
66:
67: class NeuestesFrameStream:
68:     def __init__(self, kamera_client: Esp32KameraClient) -> None:
69:         self.kamera_client = kamera_client
70:         self._sperre = threading.Lock()
71:         self._stop_ereignis = threading.Event()
72:         self._thread: threading.Thread | None = None
73:         self._capture: Any = None
74:         self._neuestes_frame: np.ndarray | None = None
75:         self._neueste_sequenz = 0
76:         self.letzter_fehler = ""
77:         self.quell lese_ms = 0.0
78:
79:     def start(self) -> None:
80:         if self._thread is not None:
81:             return
82:         self._thread = threading.Thread(target=self._lauf, daemon=True)
83:         self._thread.start()
84:
85:     def _lauf(self) -> None:
86:         aufeinanderfolgende_fehler = 0
87:
88:         while not self._stop_ereignis.is_set():
```

```

89:         if self._capture is None or not self._capture.isOpened():
90:             self._capture = self.kamera_client.stream_oeffnen()
91:             if not self._capture.isOpened():
92:                 self.letzter_fehler = (
93:                     f"{self.kamera_client.config.stream_url} konnte nicht geoffnet werden"
94:                 )
95:                 time.sleep(1.0)
96:                 continue
97:             aufeinanderfolgende_fehler = 0
98:
99:         lese_start = time.perf_counter()
100:        erfolgreich, frame = self._capture.read()
101:        lese_ms = (time.perf_counter() - lese_start) * 1000.0
102:        self.quell_lese_ms = (
103:            lese_ms
104:            if self.quell_lese_ms == 0.0
105:            else self.quell_lese_ms * 0.8 + lese_ms * 0.2
106:        )
107:
108:        if not erfolgreich or frame is None:
109:            aufeinanderfolgende_fehler += 1
110:            if aufeinanderfolgende_fehler >= 10:
111:                self.letzter_fehler = "Stream hangt. Verbindung wird neu aufgebaut..."
112:                self._capture.release()
113:                self._capture = None
114:                time.sleep(0.25)
115:                continue
116:
117:            aufeinanderfolgende_fehler = 0
118:            self.letzter_fehler = ""
119:            with self._sperr:
120:                self._neuestes_frame = frame
121:                self._neueste_sequenz += 1
122:
123:        def neuestes_lesen(
124:            self, letzte_sequenz: int
125:        ) -> tuple[int, np.ndarray | None]:
126:            with self._sperr:
127:                if (
128:                    self._neuestes_frame is None
129:                    or self._neueste_sequenz == letzte_sequenz
130:                ):
131:                    return letzte_sequenz, None
132:            return self._neueste_sequenz, self._neuestes_frame.copy()

```

```
133:
134:     def stoppen(self) -> None:
135:         self._stop_ereignis.set()
136:         if self._thread is not None:
137:             self._thread.join(timeout=2.0)
138:         if self._capture is not None:
139:             self._capture.release()
140:
141:
142: Esp32CameraClient = Esp32KameraClient
143: LatestFrameStream = NeuestesFrameStream
```